

```
"""
```

```
tensor_bridge.py - Live tensor tracking layer for the UMA pipeline.
```

```
Maps the full  $T_{\mu\nu} \leftrightarrow G_{\mu\nu}^{(1)}$  bridge at every step.
```

```
PHYSICS:
```

```
    Tμν(x,y,t) - MSR stress-energy tensor from  $\psi$  and  $\hat{\psi} = -dH/d\psi$ *  
    hμν(x,y,t) - metric perturbation derived from T00 via Poisson equation  
                   $\nabla^2 h_{\mu\nu} = -16\pi G \cdot T_{\mu\nu}$  (linearized 2+1D Einstein)  
                  Solved via FFT:  $h(k) = T(k) / (-|k|^2/16\pi G)$   
    Gμν(1)[h] - linearized Einstein tensor from h  
    κ(t)       - proportionality constant:  $T = \kappa \cdot G$   
    residual(t) -  $||T - \kappa G||_F \rightarrow 0$  at closure means Einstein eq satisfied
```

```
CLOSURE CONDITION:
```

```
    Semantic closure (friction  $\rightarrow 0$ ) AND Einstein residual  $\rightarrow 0$   
    are now the same physical event measured two ways.
```

```
"""
```

```
import numpy as np  
import math  
from dataclasses import dataclass  
from typing import List, Optional
```

```
# -----  
# Tensor record  
# -----
```

```
@dataclass
```

```
class TensorRecord:
```

```
    t: float  
    T: np.ndarray      # (2,2) spatially-averaged stress tensor  
    G: np.ndarray      # (2,2) linearized Einstein tensor  
    h: np.ndarray      # (2,2) metric perturbation (spatial mean)  
    kappa: float        #  $T = \kappa * G$   
    residual_norm: float #  $||T - \kappa * G||_F$   
    relative_residual: float # residual /  $||T||$   
    einstein_satisfied: bool # residual < threshold
```

```
# -----  
# Core computations  
# -----
```

```

def _stress_tensor_spatial(psi: np.ndarray, psi_hat: np.ndarray,
                            dx: float = 1.0) -> np.ndarray:
    """
    Compute  $T_{\mu\nu}(x,y)$  as a  $(2,2,N,N)$  spatial tensor field.


$$T_{\mu\nu} = \partial_\mu \hat{\psi} \cdot \partial_\nu \psi + \partial_\nu \hat{\psi} \cdot \partial_\mu \psi - \eta_{\mu\nu} (\partial^\alpha \hat{\psi} \cdot \partial_\alpha \psi)$$


    Uses real parts for the geometric (metric-coupled) component.
    """
    N = psi.shape[0]
    # Gradients
    dpsi = [np.gradient(psi.real, dx, axis=i) for i in range(2)]
    dpsihat = [np.gradient(psi_hat.real, dx, axis=i) for i in range(2)]

    T = np.zeros((2, 2, N, N))
    for mu in range(2):
        for nu in range(2):
            T[mu, nu] = dpsihat[mu] * dpsi[nu] + dpsihat[nu] * dpsi[mu]

    # Trace term:  $\eta_{\mu\nu} \cdot \partial^\alpha \hat{\psi} \partial_\alpha \psi$ 
    trace = sum(dpsihat[i] * dpsi[i] for i in range(2))
    T[0, 0] -= trace
    T[1, 1] -= trace

    return T

```

```

def _metric_from_T00(T_spatial: np.ndarray, G_newton: float = 1.0,
                     dx: float = 1.0) -> np.ndarray:
    """
    Derive metric perturbation  $h_{\mu\nu}$  from  $T_{00}$  via the linearized
    2+1D Einstein (Poisson) equation:


$$\nabla^2 h_{\mu\nu} = -16\pi G \cdot T_{\mu\nu}$$


    Solved via FFT. Returns spatial  $h_{\mu\nu}$  as  $(2,2,N,N)$ .
    """
    N = T_spatial.shape[-1]
    kx = 2 * np.pi * np.fft.fftfreq(N, d=dx)
    KX, KY = np.meshgrid(kx, kx)
    k2 = KX**2 + KY**2
    k2[0, 0] = 1.0 # avoid division by zero (DC mode = gauge freedom)

    h = np.zeros_like(T_spatial)
    for mu in range(2):

```

```

    for nu in range(2):
        T_k = np.fft.fft2(T_spatial[mu, nu])
        h_k = -16 * math.pi * G_newton * T_k / k2
        h_k[0, 0] = 0.0 # fix gauge: zero DC component
        h[mu, nu] = np.fft.ifft2(h_k).real

return h

def _einstein_tensor_from_h(h_spatial: np.ndarray, dx: float = 1.0) -> np.ndarray
    """
    Linearized Einstein tensor  $G_{\mu\nu}^{(1)}[h]$  as  $(2,2,N,N)$ .


$$G_{\mu\nu}^{(1)} = (1/2) (\partial^\alpha \partial_\mu h_{\alpha\nu} + \partial^\alpha \partial_\nu h_{\alpha\mu} - \square h_{\mu\nu} - \partial_\mu \partial_\nu h - \eta_{\mu\nu} (\partial^\alpha \partial^\beta h_{\alpha\beta} - \square h))$$


    In 2+1D spatial (static limit): simplifies to the 2D Laplacian form.
    """
    N = h_spatial.shape[-1]
    G = np.zeros_like(h_spatial)

    h_trace = h_spatial[0, 0] + h_spatial[1, 1]

    def lap(f):
        """2D Laplacian via finite differences."""
        return (np.roll(f, 1, 0) + np.roll(f, -1, 0) +
                np.roll(f, 1, 1) + np.roll(f, -1, 1) - 4*f) / dx**2

    def grad2(f, mu, nu):
        """ $\partial_\mu \partial_\nu f$ ."""
        df = np.gradient(f, dx, axis=mu)
        return np.gradient(df, dx, axis=nu)

    # Divergence of h:  $\partial^\alpha h_{\alpha\nu}$ 
    div_h = [
        sum(np.gradient(h_spatial[mu, nu], dx, axis=mu) for mu in range(2))
        for nu in range(2)
    ]

    # Double divergence:  $\partial^\alpha \partial^\beta h_{\alpha\beta}$ 
    div2_h = sum(
        grad2(h_spatial[mu, nu], mu, nu)
        for mu in range(2) for nu in range(2)
    )

    for mu in range(2):

```

```

    for nu in range(2):
        term1 = np.gradient(div_h[nu], dx, axis=mu)
        term2 = np.gradient(div_h[mu], dx, axis=nu)
        term3 = -lap(h_spatial[mu, nu])
        term4 = -grad2(h_trace, mu, nu)
        eta_mn = 1.0 if mu == nu else 0.0
        term5 = -eta_mn * (div2_h - lap(h_trace))
        G[mu, nu] = 0.5 * (term1 + term2 + term3 + term4 + term5)

    return G

def _spatial_mean(tensor: np.ndarray) -> np.ndarray:
    """Average (2,2,N,N) tensor over spatial dims → (2,2)."""
    return tensor.mean(axis=(-2, -1))

# -----
# TensorBridge – live tracker
# -----

class TensorBridge:
    """
    Tracks  $T_{\mu\nu}$ ,  $h_{\mu\nu}$ ,  $G_{\mu\nu}^{(1)}$ ,  $\kappa$ , and residual at every pipeline step.

    Usage::

        bridge = TensorBridge(cfg)
        # Each step:
        record = bridge.update(psi, psi_hat, t)
        # At closure:
        bridge.is_einstein_satisfied # True when residual → 0
    """

    def __init__(self, cfg=None, residual_threshold: float = 0.1,
                 dx: float = None, window: int = 30):
        kT = cfg.generic.kT if cfg else 0.005
        N = cfg.grid.N if cfg else 32
        lam = cfg.generic.lam if cfg else 0.4
        self.dx = dx if dx else 1.0 / N
        self.threshold = residual_threshold
        self.window = window # rolling average window for <T_munu>
        self.records: List[TensorRecord] = []
        self._T_buffer: List[np.ndarray] = [] # raw T history for averaging

    def update(self, psi: np.ndarray, psi_hat: np.ndarray,
              t: float) -> TensorRecord:

```

```

"""
Compute  $T_{\mu\nu}$ ,  $h_{\mu\nu}$ ,  $G_{\mu\nu}^{(1)}$ ,  $\kappa$ , residual for this step.
"""

# 1. Stress tensor (spatial)
T_spatial = _stress_tensor_spatial(psi, psi_hat, self.dx)

# 2. Metric perturbation from T via Poisson
h_spatial = _metric_from_T00(T_spatial, G_newton=1.0, dx=self.dx)

# 3. Einstein tensor from metric
G_spatial = _einstein_tensor_from_h(h_spatial, self.dx)

# 4. Spatially average to (2,2) matrices
T_inst = _spatial_mean(T_spatial)
G      = _spatial_mean(G_spatial)
h      = _spatial_mean(h_spatial)

# Time-average T over rolling window (<T_munu> satisfies Einstein eq)
self._T_buffer.append(T_inst)
if len(self._T_buffer) > self.window:
    self._T_buffer.pop(0)
T = np.mean(self._T_buffer, axis=0) # <T_munu>

# 5. Proportionality constant  $\kappa$ :  $\langle T \rangle = \kappa \cdot G$ 
T_norm = np.linalg.norm(T)
G_norm = np.linalg.norm(G)
if G_norm > 1e-15:
    #  $\kappa = \langle T, G \rangle / ||G||^2$  (least squares)
    kappa = float(np.sum(T * G) / (G_norm**2))
else:
    kappa = 0.0

# 6. Residual
residual_norm = float(np.linalg.norm(T - kappa * G))
relative_res  = residual_norm / (T_norm + 1e-15)
satisfied     = relative_res < self.threshold

rec = TensorRecord(
    t=t, T=T, G=G, h=h,
    kappa=kappa,
    residual_norm=residual_norm,
    relative_residual=relative_res,
    einstein_satisfied=satisfied,
)
self.records.append(rec)
return rec

```

```

@property
def is_einstein_satisfied(self) -> bool:
    return bool(self.records and self.records[-1].einstein_satisfied)

@property
def residual_history(self) -> np.ndarray:
    return np.array([r.relative_residual for r in self.records])

@property
def kappa_history(self) -> np.ndarray:
    return np.array([r.kappa for r in self.records])

def summary(self) -> dict:
    if not self.records:
        return {"status": "no_data"}
    final = self.records[-1]
    rh = self.residual_history
    return {
        "T_final":          final.T.tolist(),
        "G_final":          final.G.tolist(),
        "h_final":          final.h.tolist(),
        "kappa_final":      round(final.kappa, 6),
        "residual_norm_final": round(final.residual_norm, 8),
        "relative_residual": round(final.relative_residual, 6),
        "einstein_satisfied": final.einstein_satisfied,
        "residual_min":     round(float(rh.min()), 6),
        "residual_mean_last10": round(float(rh[-10:].mean()), 6),
        "n_steps":          len(self.records),
    }

```